

Kaggle Challenge Report

Multi-Date Remote Sensing Change Detection

Louis Jamin
Julian Debes
Hugo Fillet
Théophile Lahaussais

February 17, 2025

1 Feature Engineering

Our dataset contained polygons with associated statuses on five different dates, plus other urban and geographic descriptors. We removed a small number of rows containing missing values (NaN), since there were few, and they did not affect model performance significantly.

Encoding Categorical Variables. We tried a full *OneHotEncoder* for `urban_type`, `geography_type`, and each `change_status_date`, but it led to many sparse features. We switched to `pd.get_dummies` for fewer categories and dropped columns that seemed redundant. This kept relevant information while reducing dimensionality.

Date to Numeric Conversion. Each `date0`–`date4` was split into numerical features: year, month, day, weekday, and elapsed days from the earliest date. We assumed absolute time might matter (e.g., older vs. newer construction), and relative time could help identify how rapidly an area changed.

Geometric Features. We computed each polygon’s `area` and `perimeter`. Large-scale regions might indicate industrial or commercial expansions, while smaller ones could be residential.

Discarded Columns. After extracting `area` and `perimeter`, we dropped `geometry`. We also removed columns like `geography_type` when they produced too many dummy variables of negligible importance. In practice, we tested various combinations of encodings to see which performed best during cross-validation.

Feature Selection. Initially, we had around 108 features. We experimented with `SelectKBest` (chi-square) to evaluate how many to keep. Figure 1 below shows how cross-validation accuracy varied with the number of selected features. We later confirmed that keeping 81 features retained about 95% of the predictive power.

We also inspected feature importances from an `XGBoost` model. Figure 2 shows the top 108 features (in descending order of importance). From this, we concluded that eliminating the least important columns helped speed up training without hurting results.

Influence of Classifier Choice. K-nearest neighbors (k-NN) and logistic regression typically dislike high-dimensional data, so limiting the number of encoded variables was crucial. Tree-based models like `RandomForestClassifier` and `XGBoost` handled the feature space more gracefully, though

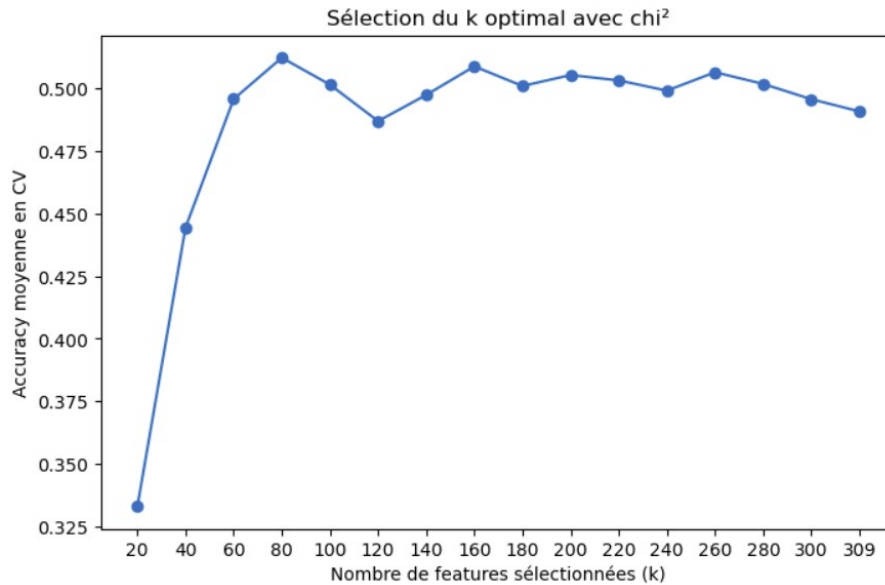


Figure 1: Cross-validation accuracy as a function of number of selected features.

we still benefited from trimming irrelevant columns.

2 Model Tuning and Comparison

We experimented with multiple classifiers and used cross-validation to estimate their generalization capabilities.

k-Nearest Neighbors (k-NN)

We began with a simple baseline. Although k-NN is easy to understand, it achieved around 40% accuracy. The high dimensionality likely caused it to overfit or yield large distance computation times. Even reducing feature count didn't push it far beyond 40–42%.

Random Forest

`RandomForestClassifier` was more robust, boosting accuracy to about 50–52%. We tested `n_estimators` up to 300 and varied `max_depth`. This improved over k-NN but eventually plateaued, suggesting that a stronger approach was needed. Random forests are also prone to many similar trees if hyperparameters are not tuned carefully.

Boosting (XGBoost)

Given the modest gains with random forests, we turned to `XGBoost`, which often excels on tabular data. We used a `RandomizedSearchCV` over:

- `n_estimators`: [100, 200, 300, 400, 500]
- `max_depth`: [3, 5, 7, 10]
- `learning_rate`: [0.01, 0.05, 0.1, 0.2, 0.3]
- `subsample`, `colsample_bytree`, `gamma`, etc.

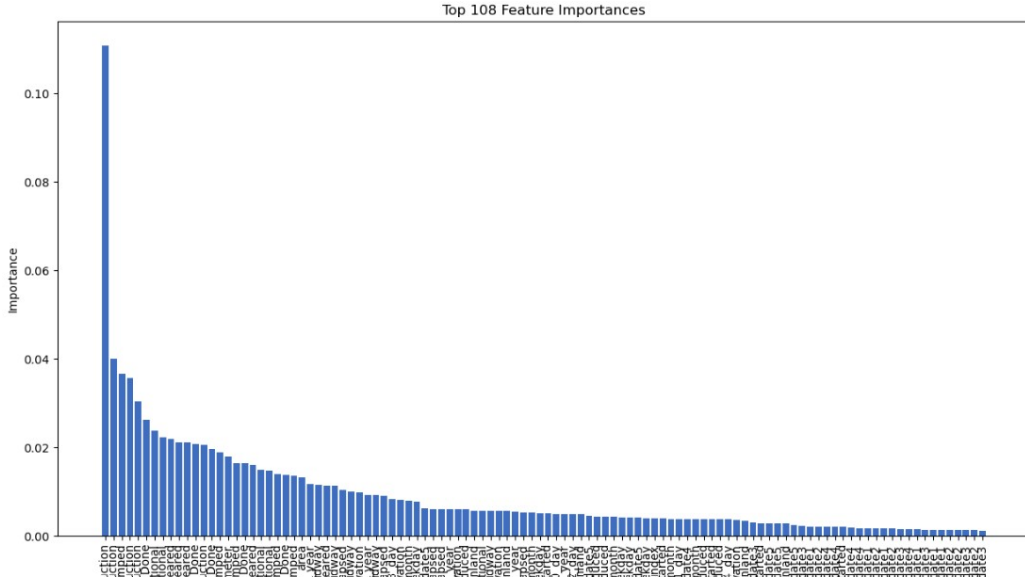


Figure 2: Top 108 feature importances from the tuned XGBoost model.

We ran 50 iterations of random search, which took about 12 hours. This careful tuning yielded a best cross-validated accuracy of around 62%. Cross-validation also helped prevent overfitting by finding suitable values for `max_depth` and `learning_rate`. We then used this tuned model for predictions on the test set and our Kaggle submission.

Other Classifiers

We briefly considered SVC (Support Vector Classifier) and logistic regression:

- **SVM:** Potentially powerful with the right kernel, but training becomes expensive in high dimensions. We would need further feature reduction or kernel approximations to make it scalable.
- **Logistic Regression:** Very fast, but limited capacity to capture non-linear boundaries in this dataset.

We also tried a `StackingClassifier` with random forest and XGBoost. Initial trials were promising, but we have not done a full hyperparameter search. In principle, a well-tuned stacking ensemble could go beyond 62%.

Performance Summary:

- k-NN: $\sim 40\%$
- Random Forest: 50–52%
- XGBoost (tuned): 62%

Our final Kaggle score reflected these cross-validated results.

3 Further Improvements

Feature Engineering. Additional time-based features (e.g., intervals between `date0` and `date1`) could reveal more about the pace of development. Spatial context features like distance to highways or city centers might also improve accuracy.

Hyperparameter Optimization. We relied on `RandomizedSearchCV`. A Bayesian optimization framework or a more extensive random search might find better XGBoost configurations in fewer iterations.

More Ensembling. Beyond stacking random forest and XGBoost, we could try blending multiple gradient-boosting models (including LightGBM or CatBoost) with different random seeds. This often yields marginal yet valuable gains.

Dimensionality Reduction. Although we cut from 108 to 81 features, advanced methods like PCA or autoencoders might reduce noise. This could help if we try more complex models, such as SVMs or neural networks, which struggle with many columns.

Conclusion

Our pipeline extracted temporal and geometric features, encoded relevant categorical variables, and used selective reduction to avoid unnecessary complexity. Among several classifiers, XGBoost stood out, achieving a cross-validated accuracy of about 62%. We see further potential in advanced ensembling, deeper hyperparameter searches, and more domain-specific feature engineering. By building on these approaches, we believe we can push performance higher while providing meaningful insights into how urban areas evolve over time.